

Exceptions Interview Questions

C++ Exception Handling — ամբողջական ժողովածու

Այս ֆայլը exception-ների interview հարցերի ժողովածու է: Երեք մաս.

Մաս A — 30 multiple-choice: Մաս B — 25 բաց (open) հարց: Մաս C — 15 tricky code / «ի՞նչ կլինի»:

Բոլոր պատասխանները **միայն վերջում** են (Answer Key A, B, C):

ՄԱՍ A — Multiple Choice (30)

Q1

Exception ի՞նչ է: - (a) memory leak - (b) error handling մեխանիզմ (throw/try/catch) - (c) thread - (d) pointer

Q2

throw ի՞նչ է անում: - (a) catch - (b) exception նետ - (c) function կանչում - (d) return

Q3

try block ինչի՞ համար: - (a) loop - (b) կող որ կարող է throw անել (պահպանված) - (c) thread - (d) lock

Q4

catch-ի կարգը: - (a) պատահական - (b) վերևից վար (կոնկրետից general) - (c) վարից վերև - (d) չկա կարգ

Q5

catch(...) ի՞նչ է: - (a) syntax error - (b) catch-all (ամեն ինչ) - (c) միայն int - (d) rethrow

Q6

Stack unwinding ինչ է: - (a) stack overflow - (b) throw-ի ժամանակ հետ call stack, local dtor-ներ կանչվում - (c) memory ազատում - (d) thread

Q7

Stack unwinding-ի ժամանակ ինչ է կանչվում: - (a) constructor-ներ - (b) local object-ների destructor-ներ - (c) main - (d) ոչինչ

Q8

catch by value-ի խնդիր: - (a) արագ - (b) slicing + copy - (c) leak - (d) ոչ մի

Q9

Ճիշտ catch ձև: - (a) catch(exception e) - (b) catch(const exception& e) - (c) catch(exception* e) - (d) catch(exception&& e)

Q10

Destructor-ից throw անելը: - (a) OK - (b) վտանգավոր - std::terminate (unwinding-ի ժամանակ) - (c) արագ - (d) leak

Q11

std::exception-ի հիմնական մեթոդը: - (a) message() - (b) what() - (c) error() - (d) msg()

Q12

vector::at() out of range -> ? - (a) UB - (b) throw std::out_of_range - (c) nullptr - (d) 0

Q13

new անհաջողություն -> ? - (a) nullptr - (b) throw std::bad_alloc - (c) crash - (d) 0

Q14

noexcept ինչ է նշանակում: - (a) չի throw - (b) միշտ throw - (c) noexcept function - (d) atomic

Q15

noexcept function throw անելիս: - (a) OK - (b) std::terminate - (c) catch - (d) return

Q16

Custom exception ինչպես: - (a) throw int - (b) ժառանգել std::exception, override what() - (c) throw string - (d) չկարելի

Q17

Strong exception guarantee: - (a) չկա leak - (b) throw-ի դեպքում վիճակ չի փոխվում (rollback) - (c) երբեք throw - (d) crash

Q18

No-throw guarantee: - (a) noexcept - երբեք չի throw - (b) rollback - (c) basic - (d) leak

Q19

RAII-ի կապը exception-ի հետ: - (a) ոչ մի - (b) exception-safe cleanup (unwinding -> dtor) - (c) արագ - (d) leak

Q20

std::out_of_range որի տակ է: - (a) runtime_error - (b) logic_error - (c) bad_alloc - (d) exception ուղիղ

Q21

Rethrow ինչպես: - (a) throw e; - (b) throw; (անանց argument) - (c) return - (d) catch

Q22

Exception normal control flow-ի համար: - (a) այո, միշտ - (b) ոչ (թանկ է, դանդաղ) - (c) միայն loop - (d) միշտ

Q23

catch(const exception&) բռնում է bad_alloc: - (a) ոչ - (b) այո (bad_alloc ժառանգում exception-ից) - (c) միայն runtime - (d) CE

Q24

throw-ից հետո կողը: - (a) կատարվում - (b) չի կատարվում (control catch է գնում) - (c) return - (d) loop

Q25

Exception-ի overhead (չկա throw): - (a) մեծ - (b) գրեթե zero (zero-cost, երբ չկա throw) - (c) 2x - (d) atomic

Q26

Multiple catch - որ մեկն է բռնվում: - (a) վերջինը - (b) առաջին համապատասխան (վերևից) - (c) բոլորը - (d) պատահական

Q27

logic_error vs runtime_error: - (a) նույնը - (b) logic (կանխատեսելի, programming), runtime (աշխատանքի) - (c) logic ավելի վատ - (d) չկա տարբերություն

Q28

what()-ի return type: - (a) string - (b) const char* - (c) int - (d) void

Q29

Constructor-ից throw անելիս object-ը: - (a) ստեղծվում - (b) չի ստեղծվում (dtor չի կանչվում իր համար) - (c) leak - (d) crash

Q30

Exception vs error code: - (a) նույնը - (b) exception՝ ավտոմատ propagate, չի կարող անտեսվել; error code՝ ձեռքով ստուգում - (c) error code ավելի լավ միշտ - (d) չկա տարբերություն

ՄԱՍ B — Open Questions (25)

1. Exception ի՞նչ է, ինչու է պետք:
2. try/catch/throw ինչպես են աշխատում:
3. Stack unwinding մանրամասն:
4. Ինչո՞ւ RAII-ն exception-safe է:
5. catch by value vs by reference:
6. Ինչո՞ւ destructor-ից չի կարելի throw:
7. std::exception հիերարխիա:
8. what() մեթոդը:
9. Custom exception ինչպես ես սարքում:
10. Exception safety guarantees (4 մակարդակ):

11. noexcept ինչ է, ինչու է պետք:
12. noexcept move constructor-ի կարևորությունը (STL):
13. catch(...) ինչպես ու երբ:
14. Multiple catch - կարգը:
15. Rethrow (throw;):
16. Exception vs error code:
17. Exception-ի overhead (zero-cost model):
18. bad_alloc, bad_cast, out_of_range:
19. logic_error vs runtime_error:
20. Constructor-ից throw - object-ի վիճակը:
21. Ինչո՞ւ exception normal flow-ի համար չէ:
22. Exception neutrality ինչ է:
23. try-catch-ի performance ազդեցություն:
24. Exception-ներ thread-ների միջև (չենք propagate):
25. Function-try-block ինչ է:

ՄԱՍ C — Tricky Code (15)

C1

```
try { throw std::runtime_error("x"); }  
catch (const std::exception& e) { cout << "caught"; }
```

- (a) caught (runtime_error ժառանգում exception)
- (b) չի բռնվի
- (c) CE
- (d) terminate

C2

```
try { throw 42; }  
catch (const std::exception& e) { cout << "ex"; }
```

- (a) ex
- (b) terminate (int չի ժառանգում exception, չկա catch)
- (c) 42
- (d) OK

C3

```
struct A { ~A(){ cout << "~A "; } };  
void f() { A a; throw std::runtime_error("x"); }  
try { f(); } catch(...) { cout << "caught"; }
```

- (a) caught
- (b) ~A caught (stack unwinding -> dtor)
- (c) caught ~A
- (d) terminate

C4

```
void f() noexcept { throw std::runtime_error("x"); }  
f();
```

- (a) caught
- (b) std::terminate (noexcept + throw)
- (c) OK
- (d) return

C5

```
try { throw std::out_of_range("x"); }  
catch (const std::logic_error& e) { cout << "logic"; }  
catch (const std::exception& e) { cout << "ex"; }
```

- (a) logic (out_of_range ժառանգում logic_error)
- (b) ex
- (c) երկուսն
- (d) terminate

C6

```
try { throw std::runtime_error("x"); }  
catch (const std::exception& e) { cout << "ex "; }  
catch (const std::runtime_error& e) { cout << "rt"; }
```

- (a) rt
- (b) ex (առաջին համապատասխան, վերևից)

- (c) երկուսն
- (d) CE

C7

```
struct A { ~A() { throw 1; } }; // dtor throw
void f() { A a; throw 2; }      // երկրորդ throw unwinding-ի ժամանակ
```

- (a) caught
- (b) std::terminate (2 throw միասին)
- (c) OK
- (d) 1

C8

```
std::vector<int> v(3);
try { cout << v.at(10); }
catch (const std::out_of_range& e) { cout << "oor"; }
```

- (a) garbage
- (b) oor
- (c) UB
- (d) 0

C9

```
std::vector<int> v(3);
cout << v[10];           // operator[]
```

- (a) throw out_of_range
- (b) undefined behavior (operator[] չի ստուգում)
- (c) oor
- (d) 0

C10

```
void f() {  
    try { throw std::runtime_error("x"); }  
    catch (const std::exception& e) { throw; }    // rethrow  
}  
try { f(); } catch (const std::exception& e) { cout << "outer"; }
```

- (a) outer (rethrow -> դուրս propagate)
- (b) ոչինչ
- (c) terminate
- (d) CE

C11

```
catch (std::exception e) { }    // by value
```

Derived exception-ի դեպքում: - (a) OK, ամբողջ - (b) slicing (derived մասը կտրվում) - (c) CE - (d) leak

C12

```
int* p = new int[10000000000000L];
```

Անհաջողության դեպքում: - (a) nullptr - (b) throw std::bad_alloc - (c) 0 - (d) crash առանց exception

C13

```
try { throw MyException("x"); }    // MyException : std::exception  
catch (const std::exception& e) { cout << e.what(); }
```

- (a) x (polymorphic what())
- (b) չի բռնվի
- (c) CE
- (d) terminate

C14

```
struct A {  
    A() { throw std::runtime_error("ctor"); }  
    ~A() { cout << "~A"; }  
};  
try { A a; } catch(...) { cout << "caught"; }
```

- (a) ~A caught
- (b) caught (ctor throw -> object չի ստեղծվի -> dtor Չի կանչվի)
- (c) terminate
- (d) ~A

C15

```
void f() { std::string s = "hi"; throw 1; }  
try { f(); } catch(int) { cout << "caught"; }
```

S -ի հետ ինչ: - (a) leak - (b) ավտոմատ ազատվում (stack unwinding -> s dtor) -> caught - (c) UB - (d) terminate

ANSWER KEY A

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-b · 14-a · 15-b · 16-b · 17-b · 18-a · 19-b · 20-b 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

ANSWER KEY B (Համառոտ)

1. Error handling մեխանիզմ; throw/try/catch; normal հոսքը ու error handling առանձին, ավտոմատ propagate:
2. throw exception նետ; try պահպանված սեկցիա; catch բռնում տեսակով (վերնից վար):
3. throw-ի ժամանակ C++-ը հետ է գնում call stack-ով catch գտնելու; այդ ընթացքում local object-ների dtor-ներ կանչվում:
4. Stack unwinding -> RAII object-ների dtor-ներ ավտոմատ կանչվում -> resource cleanup (չկա leak):

5. By value' slicing (derived կտրվում) + copy. By reference (const&)' polymorphic, չկա slicing (ճիշտ):
 6. Unwinding-ի ժամանակ (արդեն մեկը throw) երկրորդ throw -> std::terminate. dtor պետք noexcept:
 7. std::exception (what()); logic_error (invalid_argument, out_of_range), runtime_error (overflow), bad_alloc, bad_cast:
 8. virtual const char* what() const noexcept; error message:
 9. Ժառանգել std::exception, override what() որ վերադարձնի message:
 10. No-throw (noexcept), strong (rollback), basic (չկա leak), none. RAII -> առնվազն basic:
 11. Խոստում որ function չի throw; compiler optimization + STL move; throw անելիս -> terminate:
 12. move ctor noexcept լինելու դեպքում STL (vector resize) օգտագործում է move (ոչ copy) -> արագ + strong guarantee:
 13. catch(...) ամեն ինչ բռնում; երբ տիպը անհայտ է կամ catch-all cleanup պետք:
 14. Վերևից վար; առաջին համապատասխան catch-ը (դրա համար կոնկրետից general):
 15. throw; (առանց argument) catch-ի մեջ -> նույն exception-ը դուրս propagate:
 16. Exception ավտոմատ propagate, չի կարող լռելի անտեսվել; error code ձեռքով ամեն անգամ ստուգում (հեշտ մոռանալ):
 17. Zero-cost model' չկա throw -> գրեթե zero overhead; throw-ի ժամանակ թանկ (բայց հազվադեպ):
 18. bad_alloc (new fail), bad_cast (dynamic_cast reference fail), out_of_range (at()):
 19. logic_error' կանխատեսելի programming սխալ (invalid arg); runtime_error' աշխատանքի ժամանակ (I/O, overflow):
 20. ctor throw -> object չի համարվում ստեղծված -> իր dtor ՉԻ կանչվի (բայց արդեն ստեղծված member-ներինը՝ այո):
 21. Թանկ է (դանդաղ throw-ի ժամանակ, կոդ պակաս պարզ); normal flow-ի համար return/ optional ավելի լավ:
 22. Exception neutrality' function-ը բռնած exception-ները չի «կուլ տալիս», թույլ տալիս propagate (կամ rethrow):
 23. Zero-cost' չկա throw -> ~0 overhead (modern compiler); throw path-ը դանդաղ:
 24. Exception-ներ չեն propagate thread-ների միջև; եթե thread-ից exception դուրս գա -> std::terminate. std::exception_ptr-ով կարաս փոխանցել:
 25. Function-try-block' constructor-ի initializer list-ի exception բռնելու համար (try function-ի body-ից դուրս):
-

ANSWER KEY C

C1-a · C2-b · C3-b · C4-b · C5-a · C6-b · C7-b · C8-b · C9-b · C10-a C11-b · C12-b · C13-a · C14-b · C15-b

Բացատրություններ (C)

C1 runtime_error ժառանգում std::exception -> caught: **C2** throw int -> չկա int catch (միայն exception&) -> terminate: **C3** stack unwinding -> A::~A կանչվում -> "~A caught": **C4** noexcept + throw -> std::terminate: **C5** out_of_range ժառանգում logic_error -> "logic": **C6** վերնից վար, առաջին համապատասխան (exception&) -> "ex": **C7** dtor throw unwinding-ի ժամանակ (արդեն throw 2) -> 2 throw -> terminate: **C8** at() -> throw out_of_range -> "oor": **C9** operator[] չի ստուգում -> UB: **C10** rethrow (throw;) -> դուրս propagate -> "outer": **C11** by value -> slicing: **C12** new fail -> bad_alloc: **C13** MyException : std::exception, polymorphic what() -> "x": **C14** ctor throw -> object չի ստեղծվի -> dtor Չի կանչվի -> "caught": **C15** stack unwinding -> s (string) dtor ավտոմատ -> չկա leak -> "caught":

Ընդհանուր կանոններ

```
throw/try/catch | catch վերնից վար (կոնկրետից general) | catch by REFERENCE
stack unwinding -> local dtor-ներ կանչվում (RAII cleanup)
dtor-ից ՉTHROW (terminate) | noexcept+throw -> terminate
int throw -> չկա exception& catch -> terminate
at() -> out_of_range | operator[] -> UB (չի ստուգում)
ctor throw -> object չի ստեղծվի (dtor չի կանչվի)
new fail -> bad_alloc | dynamic_cast ref fail -> bad_cast
custom: ժառանգել std::exception + override what()
rethrow: throw; (առանց argument)
```